Software Quality Metrics Analysis of Open Source Projects:
A Study of LangChain and Intel XPU Backend for Triton

Garrett Gruss

Southern Methodist University

December 5, 2025

Abstract

This report presents a study of software quality metrics pertaining to two popular open source projects: Langchain-ai/langhain and intel/intel-xpu-backend-for-trition. The intial goal of this study was to collect discrete internal quality metrics such as lines of code, cyclomatic complexity over time, the project pivoted to focus on DORA (DevOps Research and Assessment) metrics due to permissions issues. pull requests, issues, and merge patterns over a timeframe of one month were collected from GitHub project insights. These metrics were used to calculate deployment frequency, lead time for changes, change failure rate and time to restore service. This study demonstrates the value of using DORA metrics as practical indicators of software project health and team performance in open source development environments.

Contents

1 Introduction

1.1 Background

The automated measurement of software quality metrics is essential for proactively monitoring and improving development processes. As open source projects grow in complexity and importance, the ability to identify quantify software quality becomes increasingly important for maintainers, contributors, and organizations depending on these projects.

Software quality metrics are essential for driving data-driven decisions regarding the development, maintenance, and deployment of modern open source software. The predictive relationship between internal metrics such as code complexity, size, structure, and external user-facing metrics such as reliability, maintainability, and performance enables teams to use measurable code properties as leading indicators of software quality. Internal metrics are available early in the software development lifecycle, and allow teams to identify potential quality issues before they manifest as failures that directly impact external users. This predictive relationship empowers teams to take a proactive approach to corrective action to improve code quality before downstream failures occur.

The importance of routine, repeatable quality measurement can be Extended to frameworks such as Goal-Question-Metrics (GQM) and the Quality Improvement Paradigm (QIP). These frameworks formalize the process of translating organizational goals into measureable indicators, enabling corrective action and discrete quality tracking. In modern DevOps environments, team velocity can neccessitate multiple deployments per day, issue resolutions, and defect identifications. This rapid velocity and continuous integration and deployment can be modeled and tracking using metrics like the DORA framework's deployment frequency, lead time for changes, change failure rate, and time to restore service have become industry standards for assessing team performance and delivery capability. For open source projects composing of distributed teams collaborating across the globe, the automated collection of quality metrics becomes essential to manage the project's health, productivity, and code

quality while maintaning the rapid velocity that bleeding-edge projects require.

1.2 Motivation

The motiviation for this project was the need to systematically assess the software quality of popular open source projects. In analyzing established projects with highly active communities, we can identify best practices in the open source community, identify patterns that correlate with project success, provide insights into development velocity, maintainability , and reliability, and establish reusable benchmarks for software quality.

1.3 Project Objectives

The primary objectives of this project were to collect software quality metrics from two major open source projects over a timespan of one month, analyze these metrics to assess project health, compare these metrics across projects to identify trends and patterns, and provide actionable insights based on quantitative analysis.

1.4 Selected Projects

1.4.1 LangChain (langchain-ai/langchain)

LangChain is a highly popular framework for developing durable and feature rich applications powered by large language models (LLMs). The project is built in Python, has 128k stars, 20k forks, and 3,812 active contributors.

1.4.2 Intel XPU Backend for Triton (intel/intel-xpu-backend-for-triton)

Intel XPU Backend for Triton is an out-of-tree backend module for Triton designed to provide high-performance GPU computing on Intel GPUs for PyTorch and standalone usage. The project targets Intel Data Center GPU Max Series, Flex Series, and Arc GPUs, supporting Ubuntu operating systems. As a compiler backend, it uses LLVM to generate optimized

code for Intel GPUs, enabling developers to write high-performance kernels using Triton's Python-based domain-specific language. The project is actively maintained as part of Intel's deep learning ecosystem and provides an alternative backend for the Triton framework. This project has 222 stars, 78 forks, and 538 active contributors.

2 Related Work

The measurement of software quality metrics in continuous integration and deployment environments has been extensively studied from both internal and external metric perspectives. This section identifies relevant research that informs the methodology and metrics selection criteria used within this paper.

2.1 Internal Metrics and Static Analysis

Research on internal software metrics has focused on automating quality measurement within CI/CD pipelines. Zampetti et al. [1] investigated how open source projects integrate static code analysis tools (Checkstyle, PMD, FindBugs) into continuous integration workflows, finding that 97% of build warnings and 6% of build failures were triggered by style issues rather than bug detection. This demonstrates that teams use internal metrics as quality gates to enforce maintainability standards. Their finding that developers typically fix issues rather than suppress warnings validates the effectiveness of enforcing internal code quality requirements within automated pipelines.

Complementing static analysis, test coverage represents a critical internal metric that serves as a leading indicator for external quality attributes. Sakamoto et al. [2] developed the Open Code Coverage Framework (OCCF) to address inconsistencies in coverage measurement across programming languages. By abstracting common concepts using abstract syntax trees, their framework reduced implementation requirements by approximately 90% while supporting multiple coverage types (statement, decision, branch, path). This work

highlights the importance of consistent measurement definitions when establishing predictive models between internal and external metrics.

Yu et al. [3] examined metrics visualization in CI environments through interviews with 22 participants across five industrial projects. They cataloged 14 base metrics spanning version control, source code, static analysis, artifacts, testing, performance, and issue tracking. Their finding that 86% of participants found CI-generated data useful for quality evaluation, but also identified challenges with dashboard information overload, underscores the need for focused metric selection rather than comprehensive collection.

2.2 External Metrics and DevOps Performance

The DevOps Research and Assessment (DORA) metrics have emerged as industry-standard indicators of software delivery performance. Wilkes et al. [4] presented a framework for automatically extracting DORA metrics (Deployment Frequency, Lead Time for Changes, Mean Time to Recover, Change Failure Rate) from project repositories without requiring specific CI/CD tools. Their analysis of 304 popular GitHub open source projects established performance baselines: top-quartile projects achieve 116.2 deployments per year with 5.1-day lead times, while bottom-quartile projects show 6.6 deployments per year with 140.6-day lead times. These metrics align with the Putnam/Myers framework's external metrics of time, quality, and productivity, providing quantifiable measures of software delivery performance.

2.3 Traceability and Security Measurement

Beyond delivery metrics, research has addressed specialized external quality dimensions. Stahl et al. [5] documented the Eiffel framework for real-time traceability throughout CI/CD pipelines, using JSON-based event messages organized as a directed acyclic graph to capture relationships between requirements, commits, builds, tests, and deployments. This automated approach produces more reliable data than manual traceability maintenance by capturing actual relationships rather than documented intentions.

Zhuravchak et al. [6] addressed temporal security measurement by proposing a lightweight system that persists and visualizes vulnerability trends across builds. Their approach combines Trivy scanning, GitHub Actions automation, and Chart.js visualization to track how dependency updates and code modifications influence security posture over time, transforming point-in-time vulnerability reports into continuous security metrics.

3 Methodology

This project evolved through three distinct phases, each representing an operational pivot in the metric collection methodology in response to technical and time-based constraints. This section documents the evolution of this study from initial planning through final implementation, providing insight into future decision-making during the design and execution of any metrics collection activities.

3.1 Phase 1: Original Plan - Static Code Analysis via GitHub Actions

3.1.1 Initial Objectives

The original methodology aimed to collect internal quality metrics within the code by integrating custom GitHub Actions workflows into target repositories. These workflows would trigger static code analysis tools on each commit or pull request to extract time-based internal metrics. The metrics chosen for this analysis were Lines Of Code (LOC), representing the source, comment, and blank lines measured via tools like cloc of tokei. Cyclomatic Complexity would be measured code path complexity measured via radon for Python. Code Coverage would be measured via a tool like pytest-cov, computed from the percentage of code excercised by tests. Technical debt would be estimated by remediation time, as calculated from SonarQube or CodeClimate analysis. Maintainability index would be computed as a composite metrics combining complexity, LOC, and comment density.

3.1.2 Planned Implementation

The planned implementation of the static collection of metrics via workflows involved creating custom GitHub Actions workflows, installing the custom static analysis tools in the workflow environment, running the analysis on each commit or pull request, and publishing the metrics to a centralized database or observability platform.

3.1.3 Abandonment Rationale

This approach was abandonded during implementation due to its invasive nature. Adding custom GitHub actions and workflows to projects required forking repositories on a per-branch basis, resulting in excessive modification of the source code repository to collect metrics. Large projects like Langchain also have existing CI/CD pipelines, so integrating custom Actions involved risking compatability conflicts with existing workflows, which could pollute the accuracy of collected software quality metrics. Each target repository would also required custom workflows tailored to it's language, build system, and testing framework. The invasiveness of this approach required a pivot to a more observant solution.

3.2 Phase 2: First Pivot - DORA Metrics via OpenTelemetry

3.2.1 Revised Approach

To avoid modification to the target repository, the collection methdology was pivoted to utilizing the Open Telemetry collectors to scrape DORA metrics from GitHub. This non-invasive approach automated metric collection through the use of GitHub's GraphQL and REST APIs without requiring change to the target repositories. The planned system was deployed through docker Compose using an OpenTelemetry Collector container that scraped GitHub repositories every 60 seconds and an OpenObserve metrics aggregation and visualization service that collected the streamed metrics.

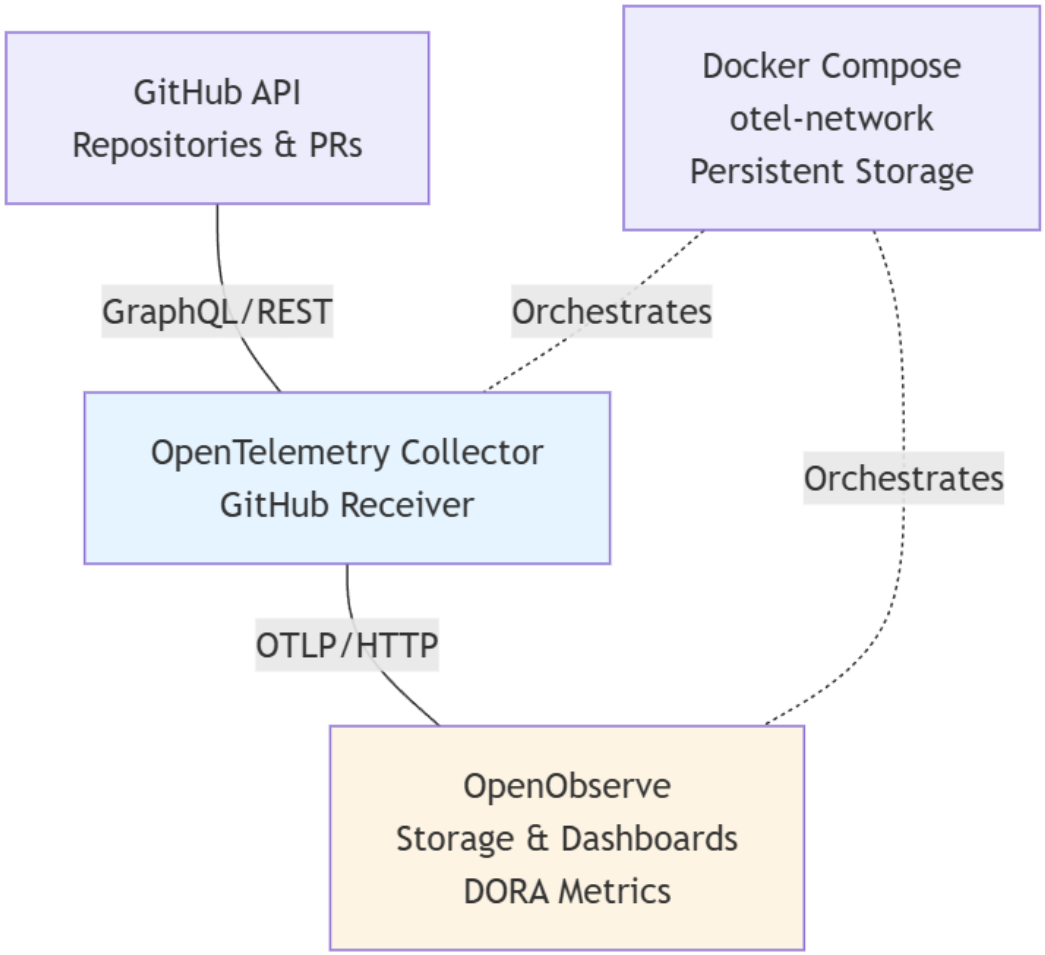

Figure 1: OpenTelemetry DORA Metrics Architecture (Phase 2 Approach)

3.2.2 Target Metrics

The GitHub receiver was configured to collect nine VCS (Version Control System) metrics conforming to OpenTelemetry semantic conventions v1.37.0:

Deployment Frequency Support:

- `vcs.change.count` - Pull request counts by state (open/merged)
- `vcs.repository.count` - Total repositories monitored

Lead Time for Changes Support:

- `vcs.change.time_to_merge` - Duration from PR creation to merge (seconds)

- `vcs.change.time_to_approval` - Duration from PR creation to approval (seconds)
- `vcs.change.duration` - Time PRs remain in open state (seconds)

Development Velocity Indicators:

- `vcs.ref.count` - Branch/tag counts per repository
- `vcs.ref.time` - Branch lifespan from creation (seconds)
- `vcs.ref.lines_delta` - Code churn (lines added/removed) vs. trunk
- `vcs.ref.revisions_delta` - Commit count ahead/behind trunk

3.2.3 Challenges Encountered

While less invasive than GitHub Actions, the OpenTelemetry approach encountered permissions issues when scraping metrics from public repositories. The OpenTelemetry collector requires a GitHub PAT token with specific organization access to **repo** and read:org. Public repositories limit provided data through unauthorized API access, prohibiting the collection of detailed PR metrics required to compute DORA. Accessing langchain-ai and intel repositories requires a GitHub PAT with organization access, approved by organization administrators.

3.3 Phase 3: Final Pivot - Manual DORA Metrics from GitHub Web UI

3.3.1 Rationale for Final Approach

Since automated collection of detailed metrics was prohibited due to permissions issues, a simplier approach of scraping metrics from the GitHub Insights page was chosen as the preferred data collection strategy. This approach leveraged publicly accessible GitHub repository insights through the web interface. While less automated and less detailed than the

scraped metrics, this approach did not require specific permisisons granted by organizational administrators.

3.3.2 Data Collection Process

Data was manually extracted from GitHub's web interface using the repository Insights tab and Pull Requests/Issues pages. For each target repository, Pull Request Events, Issue Events, and metadata were collected. This manual approach bypassed all authentication and permission barriers encountered in previous phases.

3.3.3 Data Structure

The collected data was organized into TXT files:

- `opened_requests.txt` - Timestamps of PR creation
- `merged_requests.txt` - Timestamps of PR merges
- `closed_requests.txt` - Timestamps of PR closures
- `opened_issues.txt` - Timestamps of issue creation

3.4 Analysis Methodology

3.4.1 DORA Metrics Framework

The four key DORA metrics, established by the DevOps Research and Assessment team, serve as the analytical foundation:

1. **Deployment Frequency (DF):** How often an organization successfully releases to production (proxied by merged PR rate)
2. **Lead Time for Changes (LT):** The time it takes for a commit to get into production (measured as PR open-to-merge duration)

3. **Change Failure Rate (CFR):** The percentage of deployments causing a failure in production (calculated from bug issue frequency)

4. **Time to Restore Service (MTTR):** How long it takes to recover from a failure in production (measured as bug report to fix merge duration)

3.4.2 Metric Calculation

A Python-based metrics collection pipeline was constructed that loads and preprocesses scrapped metrics data, converts them into columnar CSV data, calculates DORA metrics using industry-standard forumlas, performs statistical analysis, classifies DORA performance, and generates visualizations for each metric.

3.4.3 Deployment Frequency Calculation

$$DF = \frac{\text{Total Merged PRs}}{\text{Time Period (days)}} \tag{1}$$

3.4.4 Lead Time Calculation

For each merged PR i:

$$LT_i = T_{\text{merged}_i} - T_{\text{opened}_i} \tag{2}$$

Aggregate metric: median(LT_i)

3.4.5 Change Failure Rate Calculation

$$CFR = \frac{\text{Number of Bug Issues}}{\text{Total Deployments}} \times 100\% \tag{3}$$

Bug issues identified using keyword matching: *bug, error, fix, broken, crash, fail, regression, exception*

3.4.6 Time to Restore Calculation

For each bug-fix pair (b, f):

$$MTTR_{b,f} = T_{\text{fix merged}_f} - T_{\text{bug opened}_b} \tag{4}$$

Aggregate metric: median($MTTR_{b,f}$)

3.4.7 Performance Classification

DORA performance was classified according to the 2023 DORA State of DevOps Report standards.

Table 1: DORA Performance Level Classifications

Metric	**Elite**	**High**	**Medium**	**Low**
Deployment Frequency	Multiple/day	Weekly	Monthly	$<$ Monthly
Lead Time	$<$ 1 day	1-7 days	1-4 weeks	$>$ 1 month
Change Failure Rate	0-15%	16-30%	31-45%	$>$ 45%
Time to Restore	$<$ 1 hour	¡ 1 day	1-7 days	$>$ 1 week

4 Results

4.1 LangChain Analysis

4.1.1 Data Overview

Analysis of LangChain-ai/Langchain was conducted over a 30-day period from October 29, 2025 to November 27, 2025. During this period, 179 merged pull requests, 67 opened pull requests, 89 closed pull requests, and 69 issues were captured and analyzed.

4.1.2 Deployment Frequency

The LangChain project demonstrated high rate of deployment activity during the analysis period. The mean deployment frequency was 5.97, with a median of 8.5 deployments per day, resulting in 41.77 deployments per week on average. This deployment frequency classifies the project as an elite performer according to the DORA benchmark. This elite deployment frequency represents a mature CI/CD pipeline and team mentality resulting in multiple deploys per day.

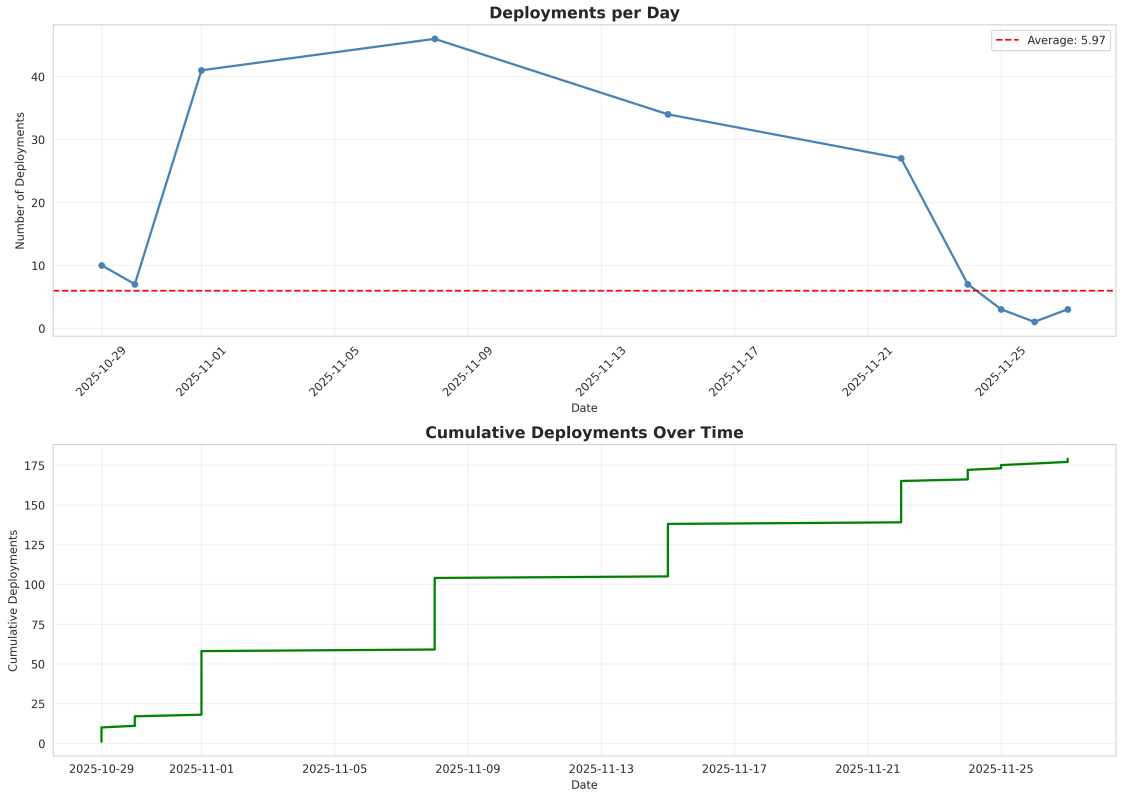

Figure 2: LangChain Deployment Frequency Over Time

4.1.3 Lead Time for Changes

lead time for changes was estimated using temporal proximity matching between opened and merged PRs. The analysis across 162 matched PR paris relvealed a median lead time of 6.00 days and a mean of 4.98 days. The distribution showed a 25th percentile of 2.00 days and a 90th percentile of 7.00 days. Five pull requests were merged within 24 hours,

representing the quickest turnaround times. According to DORA performance standards, the median lead time of 6.00 days falls within the high performance category of 1-7 days.

4.1.4 Change Failure Rate

Change failure rate was estimated by using keyword matching on failure terms such as "bug", "error", "fix", "broken", "crash", "fail", "regression", and "exception". Out of 69 collected issues, 16 were classified as bug-releated, representing 23.2% of the total population. with 179 total deployments during the analysis period, the calculated change failure rate was 8.94%. This rate falls within the elite performance category according to the DORA standard, which classifies rates between 0-15% as elite.

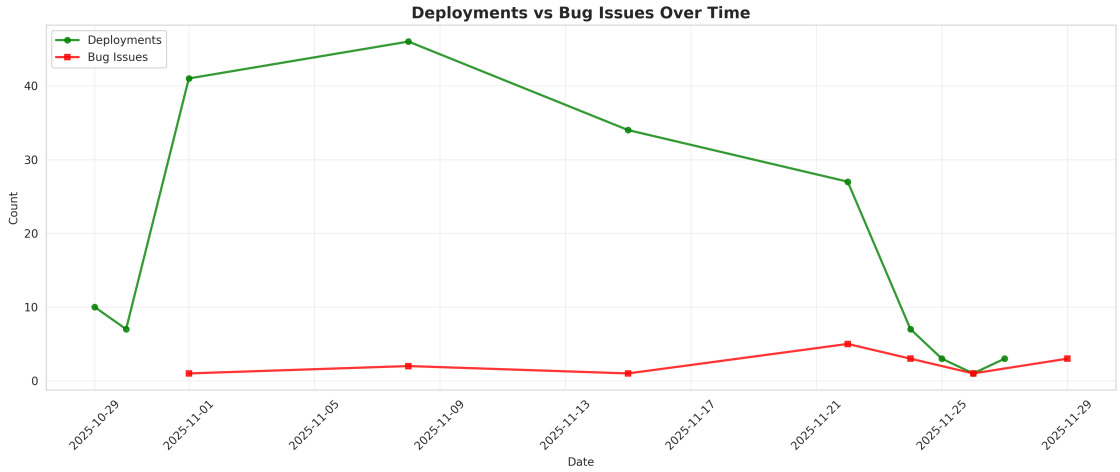

Figure 3: LangChain Deployments vs Bug Issues

4.1.5 Time to Restore Service

The time to restore service (MTTR) metric could not be calculated due to limitations occuring collecting related issues and merged pull requests. Estimation of MTTR requires collecting an opened issue, and identifying the time when the issue was resolved through a pull request and merge approval event. Future work could improve bug-fix pair identification using a properly authenticated GitHub PAT connected to an open-Telemetry GitHub scraper.

4.1.6 Overall Performance

Table 2: LangChain DORA Metrics Summary

Metric	**Value**	**Classification**
Deployment Frequency	5.97/day, 41.77/week	Elite
Lead Time for Changes	6.00 days (median)	High
Change Failure Rate	8.94%	Elite
Time to Restore Service	N/A	N/A
Overall Rating	**High Performer**	

4.2 Intel XPU Backend for Triton Analysis

4.2.1 Data Overview

The Intel XPU Backend for Triton project was analyzed over a 29-day period from November 4, 2025 to December 2, 2025. The dataset included 104 merged pull requests, 17 opened pull requests, 73 closed pull requests, and 42 opened issues.

4.2.2 Deployment Frequency

The mean deployment frequency was 3.59 deplyoments per day with a median of 9.0 deployments per day, totaling to 25.10 average deployments per week. The maximum number of deployments in a single day was 28. According to the DORA standard, this frequency classifies as an elite performer, achieving multiple deployments per day.

4.2.3 Lead Time for Changes

lead time for changes was estimated using temporal proximity matching between opened and merged PRs. However, the metrics collected were statistically insufficient, with only 3

matching pull requests identified. The limited sample size makes this metric unreliable for overall DORA assessment.

4.2.4 Change Failure Rate

Bug-related issues were identified using keyword matching. Of the total 42 issues collected, 11 were classified as defect-related, representing 26.2% of the total issue population. With 104 total deployments during the analysis period, the calculated failure rate was 10.58%. According to the DORA standard, this rate falls within the elite performance category of 0-15%.

4.2.5 Time to Restore Service

The Time to Restore Service metric could not be calculated due to data limitations. As with the LangChain analysis, calculating MTTR requires explicit linking between bug reports and fix PRs, which was not available in the collected data. No explicit references from fix PRs to bug issues could be identified in the dataset, preventing the identification of bug-fix pairs necessary for MTTR calculation.

4.2.6 Overall Performance

Table 3: Intel XPU Backend DORA Metrics Summary

Metric	**Value**	**Classification**
Deployment Frequency	3.59/day, 25.10/week	Elite
Lead Time for Changes	N/A (insufficient data)	N/A
Change Failure Rate	10.58%	Elite
Time to Restore Service	N/A	N/A
Overall Rating	**Elite Performer***	

* Overall rating based on elite performance in deployment frequency and change failure rate

4.3 Comparative Analysis

Table 4: Comparative DORA Metrics: LangChain vs Intel XPU Backend

Metric	**LangChain**	**Intel XPU**
Deployment Frequency	5.97/day (Elite)	3.59/day (Elite)
Lead Time for Changes	6.00 days (High)	N/A
Change Failure Rate	8.94% (Elite)	10.58% (Elite)
Time to Restore Service	N/A	N/A
Overall Rating	High Performer	Elite Performer

5 Discussion

5.1 Key Findings

5.1.1 Development Velocity

Both projects achieve elite-level deployment frequency: LangChain at 5.97 deployments per day, Intel XPU Backend at 3.59 per day. These rates indicate mature CI/CD pipelines supporting multiple daily releases. LangChain's 6-day median lead time balances velocity with code review thoroughness. Intel XPU Backend lacks sufficient lead time data but demonstrates efficient merge workflows through high deployment frequency.

5.1.2 Code Quality and Reliability

Both projects achieve elite-level change failure rates below the 15% threshold: LangChain at 8.94%, Intel XPU Backend at 10.58%. High deployment frequency combined with low failure rates indicates robust testing, effective code review, and comprehensive CI/CD quality gates,

demonstrating mature velocity-quality balance.

5.1.3 Incident Response

Time to restore service could not be calculated due to limitations in linking bug reports to fix PRs without authenticated API access. This gap does not diminish the validity of the three successfully measured DORA metrics.

5.2 Project Comparison

5.2.1 Similarities

Both projects achieve elite-level deployment frequency and change failure rates with daily merge activity, sustaining high velocity without compromising quality. This pattern represents open source best practices regardless of domain or size.

5.2.2 Differences

LangChain shows higher deployment frequency (5.97 vs 3.59 per day) and sufficient data for lead time analysis, while Intel XPU Backend lacks adequate PR data for lead time calculation. Notably, Intel XPU Backend achieves 60% of LangChain's deployment frequency despite having only 0.17% of the stars (222 vs 128k) and 14% of contributors (538 vs 3,812), suggesting exceptional productivity in Intel's smaller, corporate-backed team. Domain differences (AI framework vs compiler infrastructure) may influence development patterns and quality practices.

5.3 Industry Benchmarking

Both projects significantly exceed Wilkes et al.'s top-quartile benchmark of 116.2 annual deployments (0.32 per day): LangChain with 2,180 annual deployments and Intel XPU Backend with 1,310, placing both in the top percentile. Elite change failure rates (under 15%)

match DORA 2023 elite performer classification, confirming industry-leading performance.

5.4 Practical Value of DORA Metrics

DORA metrics enable rapid comparison across projects through simple calculation and standardized classification. Unlike internal code metrics requiring language-specific tooling, DORA metrics use basic temporal data from any version control system: merges per day, PR-to-merge time, and bug ratios. This facilitated direct comparison between LangChain and Intel XPU Backend despite domain, size, and architecture differences. Standardized performance tiers (elite/high/medium/low) enable immediate project assessment without baseline establishment. Practical applications include portfolio analysis, competitive benchmarking, and trend monitoring across diverse projects.

5.5 Limitations and Threats to Validity

5.5.1 Data Collection Limitations

- Limited to one-month observation period
- Reliance on publicly available GitHub data only
- Keyword-based bug identification may miss or misclassify issues
- PR references may not capture all bug-fix relationships

5.5.2 Metric Calculation Assumptions

- Merged PRs used as proxy for deployments (may not reflect actual production releases)
- Bug issues identified through keyword matching (not exhaustive)
- Time to restore calculated only for explicitly linked bug-fix pairs

5.5.3 Generalizability

- Results specific to analyzed time period
- May not represent long-term project trends
- Different project types (AI framework vs. compiler backend) have different normal patterns

6 Lessons Learned

6.1 Technical Challenges

External software quality research faces an inverse relationship between metric depth and feasibility. GitHub Actions provided rich code-level metrics but required invasive repository modifications. OpenTelemetry eliminated invasiveness but hit API authentication barriers requiring organizational approval. Manual collection from public GitHub data proved universally accessible but limited to PR/issue metadata. For external research, non-invasive observation using publicly available data remains the only pragmatic approach. DORA metrics derived from public GitHub data provide meaningful quality assessment without privileged access.

6.2 Best Practices

Design data collection assuming no repository access or authentication. Prioritize non-invasive observation using public GitHub Insights, PR history, and issue tracking. Use industry-standard frameworks like DORA for interpretability and benchmarking without custom baselines. Automate analysis pipelines rather than collection for reproducibility. Test approaches on accessible repositories before large-scale collection. Document methodological pivots to inform future researchers.

7 Conclusion

This study analyzed software quality metrics for two open source projects using DORA metrics from public GitHub data. Both LangChain (5.97 daily deployments, 8.94% failure rate) and Intel XPU Backend (3.59 daily deployments, 10.58% failure rate) achieve elite-level performance, exceeding industry benchmarks and demonstrating that high velocity does not require compromising quality.

DORA metrics provide simple, standardized measurements enabling direct comparison across projects regardless of domain, size, or architecture. This study's comparison between an AI framework and compiler infrastructure demonstrates practical applicability for portfolio analysis, competitive benchmarking, and trend monitoring. The methodology is reproducible for any public GitHub repository.

External research on production systems requires non-invasive approaches using public data. DORA metrics derived from GitHub provide meaningful quality assessment without privileged access. Future work should correlate DORA trends with internal metrics (cyclomatic complexity, test coverage, technical debt) to establish predictive relationships between deployment practices and code maintainability, enabling DORA metrics as leading indicators for quality degradation.

References

[1] F. Zampetti, S. Scalabrino, R. Oliveto, G. Canfora, and M. Di Penta, "How open source projects use static code analysis tools in continuous integration pipelines," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. Buenos Aires, Argentina: IEEE, 2017, pp. 334–344.

[2] K. Sakamoto, H. Washizaki, and Y. Fukazawa, "Open code coverage framework: A consistent and flexible framework for measuring test coverage supporting multiple programming

languages," in *2010 10th International Conference on Quality Software*. Zhangjiajie, China: IEEE, 2010, pp. 262–269.

[3] L. Yu, E. Alégroth, P. Chatzipetrou, and T. Gorschek, "A roadmap for using continuous integration environments," *Communications of the ACM*, vol. 67, no. 6, pp. 82–90, June 2024.

[4] B. Wilkes, A. M. P. Milani, and M.-A. Storey, "A framework for automating the measurement of devops research and assessment (dora) metrics," in *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. Bogotá, Colombia: IEEE, 2023, pp. 62–72.

[5] D. Ståhl, K. Hallén, and J. Bosch, "Continuous integration and delivery traceability in industry: Needs and practices," in *2016 42nd Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Limassol, Cyprus: IEEE, 2016, pp. 68–72.

[6] Y. Zhuravchak, D. Zhuravchak, A. Zhuravchak, and I. Opirskyy, "Security regression visualization in CI/CD using Trivy and GitHub Actions," in *CSDP'2025: Cyber Security and Data Protection*, Lviv, Ukraine, July 2025, pp. 272–281.

A Phase 1 Planned Configuration (GitHub Actions)

This section documents the original Phase 1 approach that involved integrating GitHub Actions workflows into target repositories for automated static code analysis and quality metric collection.

A.1 Planned GitHub Actions Workflow

The intended workflow would have been added to target repositories at `.github/workflows/quality-metr`

Listing 1: Planned quality-metrics.yml Workflow

```yaml
name: Software Quality Metrics Collection

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  quality-analysis:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install radon pytest pytest-cov ruff mypy
          pip install -r requirements.txt

      - name: Run Radon - LOC Analysis
        run: |
```

```
      radon raw . -j > metrics/radon_loc.json
      radon cc . -j > metrics/radon_complexity.json
      radon mi . -j > metrics/radon_maintainability.json

  - name: Run Pytest with Coverage
    run: |
      pytest --cov=. --cov-report=json \
             --cov-report=term \
             --junitxml=metrics/pytest_results.xml

  - name: Run Ruff Linter
    run: |
      ruff check . --output-format=json \
           > metrics/ruff_violations.json

  - name: Run Mypy Type Checker
    run: |
      mypy . --json-report metrics/mypy_report

  - name: Enforce Quality Gates
    run: |
      python .github/scripts/enforce_quality_gates.py

  - name: Publish Metrics
    uses: actions/upload-artifact@v3
    with:
      name: quality-metrics
      path: metrics/

```

```
      - name: Send to Observability Platform
        run: |
          python .github/scripts/publish_to_otel.py
        env:
          OTEL_ENDPOINT: ${{ secrets.OTEL_ENDPOINT }}
          OTEL_API_KEY: ${{ secrets.OTEL_API_KEY }}
```

A.2 Planned Quality Gate Enforcement

The workflow would have included automated quality gates to prevent merging code that failed thresholds:

Listing 2: enforce_quality_gates.py

```python
import json
import sys

# Load metrics
with open('metrics/radon_complexity.json') as f:
    complexity = json.load(f)

with open('metrics/pytest_coverage.json') as f:
    coverage = json.load(f)

with open('metrics/ruff_violations.json') as f:
    linting = json.load(f)

# Define thresholds
THRESHOLDS = {
    'max_complexity': 15,
```

```
    'min_coverage': 70,
    'max_linting_violations': 50,
    'min_maintainability': 50
}

# Check thresholds
violations = []

# Check cyclomatic complexity
for module in complexity:
    for func in module.get('functions', []):
        if func['complexity'] > THRESHOLDS['max_complexity']:
            violations.append(
                f"Function {func['name']} has complexity "
                f"{func['complexity']} (max: {THRESHOLDS['
                    max_complexity']})"
            )

# Check test coverage
total_coverage = coverage['totals']['percent_covered']
if total_coverage < THRESHOLDS['min_coverage']:
    violations.append(
        f"Test coverage {total_coverage}% is below "
        f"minimum {THRESHOLDS['min_coverage']}%"
    )

# Check linting violations
violation_count = len(linting)
if violation_count > THRESHOLDS['max_linting_violations']:
```

```
    violations.append(
        f"Linting␣violations␣{violation_count}␣exceed␣"
        f"maximum␣{THRESHOLDS['max_linting_violations']}"
    )

# Report violations
if violations:
    print("Quality␣gate␣FAILED:")
    for v in violations:
        print(f"␣␣-␣{v}")
    sys.exit(1)
else:
    print("Quality␣gate␣PASSED")
    sys.exit(0)
```

A.3 Planned Tool Configurations

A.3.1 Radon Configuration

Radon would analyze code complexity without additional configuration, but the following commands were planned:

Listing 3: Radon Analysis Commands

```
# Lines of Code analysis
radon raw . --json --output-file=radon_loc.json

# Cyclomatic Complexity
radon cc . --json --min=C --output-file=radon_complexity.json

# Maintainability Index
```

```
radon mi . --json --min=B --output-file=radon_maintainability.json

# Halstead metrics (code effort estimation)
radon hal . --json --output-file=radon_halstead.json
```

A.3.2 Pytest Configuration

Listing 4: pytest.ini

```
[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*

# Coverage settings
addopts =
    --cov=src
    --cov-report=html
    --cov-report=json
    --cov-report=term-missing
    --cov-fail-under=70
    --junitxml=junit.xml
    --verbose

# Coverage exclusions
[coverage:run]
omit =
    */tests/*
    */venv/*
```

```
    */migrations/*
    */__pycache__/*

[coverage:report]
precision = 2
show_missing = True
skip_covered = False
```

A.3.3 Ruff Configuration

Listing 5: pyproject.toml - Ruff Configuration

```
[tool.ruff]
line-length = 100
target-version = "py311"

select = [
    "E",   # pycodestyle errors
    "W",   # pycodestyle warnings
    "F",   # pyflakes
    "I",   # isort
    "C",   # flake8-comprehensions
    "B",   # flake8-bugbear
    "UP",  # pyupgrade
    "N",   # pep8-naming
]

ignore = [
    "E501",  # line too long (handled by formatter)
    "B008",  # function call in argument defaults
```

```
]

[tool.ruff.per-file-ignores]
"__init__.py" = ["F401"]  # unused imports
"tests/*.py" = ["S101"]   # allow assert statements

[tool.ruff.isort]
known-first-party = ["src"]
```

A.3.4 Mypy Configuration

Listing 6: mypy.ini

```
[mypy]
python_version = 3.11
warn_return_any = True
warn_unused_configs = True
disallow_untyped_defs = True
disallow_incomplete_defs = True
check_untyped_defs = True
no_implicit_optional = True
warn_redundant_casts = True
warn_unused_ignores = True
warn_no_return = True
strict_equality = True
show_error_codes = True

# Per-module options
[mypy-tests.*]
disallow_untyped_defs = False
```

```
18
19 [mypy-third_party_lib.*]
20 ignore_missing_imports = True
```

A.4 Planned Semantic Versioning Integration

The workflow would have used `python-semantic-release` for automated version management:

Listing 7: Semantic Release Workflow

```
1  name: Semantic Release
2
3  on:
4    push:
5      branches: [main]
6
7  jobs:
8    release:
9      runs-on: ubuntu-latest
10     steps:
11       - uses: actions/checkout@v3
12         with:
13           fetch-depth: 0
14
15       - name: Python Semantic Release
16         uses: relekang/python-semantic-release@master
17         with:
18           github_token: ${{ secrets.GITHUB_TOKEN }}
19           pypi_token: ${{ secrets.PYPI_TOKEN }}
```

Listing 8: pyproject.toml - Semantic Release Config

```toml
[tool.semantic_release]
version_variable = "src/__init__.py:__version__"
version_source = "tag"
commit_version_number = true
tag_commit = true
upload_to_pypi = false
upload_to_release = true

branch = "main"
hvcs = "github"

# Commit parsing
commit_parser = "angular"
major_on_zero = true
allow_zero_version = true

# Version bumping rules
[tool.semantic_release.commit_parser_options]
allowed_tags = [
    "feat",      # New feature (minor bump)
    "fix",       # Bug fix (patch bump)
    "docs",      # Documentation only
    "style",     # Code style changes
    "refactor",  # Code refactoring
    "perf",      # Performance improvements
    "test",      # Test additions
    "build",     # Build system changes
    "ci",        # CI configuration
```

```
]

major_tags = ["BREAKING CHANGE", "BREAKING"]
minor_tags = ["feat"]
patch_tags = ["fix", "perf"]
```

A.5 Abandonment Rationale - Detailed

This comprehensive GitHub Actions approach was abandoned due to several critical barriers:

1. **Repository Modification Complexity**
   - Adding workflows requires write access to `.github/workflows/` directory
   - Target repositories (LangChain, Intel XPU Backend) have existing CI/CD infrastructure
   - Risk of conflicts with existing workflows, branch protection rules, and required checks
   - Need to modify `.gitignore`, add `pyproject.toml`, and create metrics directories
2. **Maintainer Approval Barriers**
   - External researchers cannot submit PRs that add observability workflows for academic purposes
   - Maintainers would reject workflows that export metrics to external platforms (security/privacy)
   - Request would require detailed justification, privacy impact assessment, and legal review
   - No precedent for academic metric collection in production repositories
3. **Per-Repository Customization Requirements**

- Different projects use different languages (Python, C++, Rust), requiring tool variations
- Test frameworks vary: pytest, unittest, cargo test, gtest, etc.
- Build systems differ: setuptools, poetry, cargo, CMake, requiring custom metric extraction
- Each repository would need custom quality gate thresholds based on project maturity

4. **Fork Limitations**

   - Forking allows workflow addition but creates isolation from production development
   - Forks do not receive upstream commits in real-time, metrics become stale
   - Fork metrics do not reflect actual project health, only snapshot quality
   - Cannot access organizational secrets, runners, or internal deployment metrics

5. **Computational and Financial Costs**

   - GitHub Actions minutes are limited for free accounts (2,000 minutes/month)
   - Running comprehensive quality analysis on large codebases consumes significant minutes
   - LangChain repository has 150+ contributors with dozens of daily commits
   - Would require paid GitHub plan ($4-$21/user/month) for adequate compute quota

These constraints led to the pivot toward less invasive approaches in Phase 2 and Phase 3.

B Data Collection Scripts

The data collection process involved manually extracting data from GitHub's web interface and organizing it into CSV format. While no automated scraping scripts were used due to permission constraints, the following describes the manual collection workflow:

B.1 GitHub Web UI Navigation

For each repository (LangChain and Intel XPU Backend), data was collected through:

1. **Pull Request Data Collection**
    - Navigate to repository → Pull Requests tab
    - Filter by date range (e.g., October 29 - November 27, 2025 for LangChain)
    - Filter by state: `is:pr is:open`, `is:pr is:merged`, `is:pr is:closed`
    - Copy PR titles, numbers, and timestamps to CSV
2. **Issue Data Collection**
    - Navigate to repository → Issues tab
    - Filter by creation date within analysis window
    - Identify bug-related issues using keyword search: bug, error, fix, broken, crash, fail, regression, exception
    - Export issue titles and creation timestamps
3. **Repository Metadata**
    - Navigate to Insights → Pulse for activity summary
    - Record commit hashes and dates from commit history
    - Note relative timestamps ("2 days ago") and convert to absolute dates

B.2 CSV Data Organization

Data was organized into four CSV files per repository:

Listing 9: CSV File Structure

```
# opened_requests.csv
msg,hash,date_rel,date_abs
"Add␣authentication␣feature",#12345,"2 days ago","2025-11-27"

# merged_requests.csv
msg,hash,date_rel,date_abs
"Fix␣authentication␣bug",#12346,"1 day ago","2025-11-28"

# closed_requests.csv
msg,hash,date_rel,date_abs
"Update␣documentation",#12347,"3 hours ago","2025-11-29"

# opened_issues.csv
msg,hash,date_rel,date_abs
"Bug:␣Login␣fails␣on␣Safari",#678,"1 week ago","2025-11-22"
```

C Analysis Source Code

The complete analysis pipeline is available in `examples/dora_metrics.py`. The script implements all DORA metric calculations and generates visualizations.

C.1 Key Functions

- `load_data(filepath)`: Loads CSV data and preprocesses timestamps
  - Parses date_abs column into datetime objects

  - Handles missing or malformed timestamps
  - Returns pandas DataFrame

- `calculate_deployment_frequency(merged_df, days)`: Calculates DF metric
  - Counts merged PRs per day over analysis window
  - Computes mean, median, and weekly aggregate
  - Classifies performance: Elite (>1/day), High (weekly), Medium (monthly), Low (<monthly)

- `calculate_lead_time(opened_df, merged_df)`: Calculates LT metric
  - Matches opened PRs with merged PRs using temporal proximity
  - Computes time delta in days
  - Returns median, mean, 25th/75th/90th percentiles
  - Classifies: Elite (<1 day), High (1-7 days), Medium (1-4 weeks), Low (>1 month)

- `calculate_change_failure_rate(issues_df, merged_df)`: Calculates CFR metric
  - Identifies bug issues using keyword matching on issue titles/descriptions
  - Counts bug issues within analysis window
  - Computes ratio: (bug issues / total deployments) × 100%
  - Classifies: Elite (0-15%), High (16-30%), Medium (31-45%), Low (>45%)

- `calculate_time_to_restore(issues_df, merged_df)`: Calculates MTTR metric
  - Attempts to pair bug issues with fix PRs
  - Searches for issue references in PR titles/bodies
  - Computes time from bug opened to fix merged
  - Returns median restoration time

  - Classifies: Elite (<1 hour), High (<1 day), Medium (1-7 days), Low (>1 week)

- `generate_visualizations(metrics_dict, output_dir)`: Creates charts
  - Deployment frequency time series
  - Lead time distribution histogram
  - Change failure rate comparison (deployments vs bugs)
  - Performance classification summary

C.2 Usage Example

Listing 10: Running DORA Metrics Analysis

```
import pandas as pd
from dora_metrics import *

# Load data
opened = load_data('data/langchain/opened_requests.csv')
merged = load_data('data/langchain/merged_requests.csv')
closed = load_data('data/langchain/closed_requests.csv')
issues = load_data('data/langchain/opened_issues.csv')

# Calculate metrics
df = calculate_deployment_frequency(merged, days=30)
lt = calculate_lead_time(opened, merged)
cfr = calculate_change_failure_rate(issues, merged)
mttr = calculate_time_to_restore(issues, merged)

# Generate visualizations
metrics = {'df': df, 'lt': lt, 'cfr': cfr, 'mttr': mttr}
```

```python
generate_visualizations(metrics, 'output/langchain/')

# Print summary
print(f"Deployment Frequency: {df['mean']:.2f}/day")
print(f"Lead Time (median): {lt['median']:.2f} days")
print(f"Change Failure Rate: {cfr['rate']:.2f}%")
print(f"Time to Restore: {mttr['median']:.2f} hours")
```

D Raw Data Samples

D.1 LangChain Repository Data

Sample entries from the LangChain analysis dataset (October 29 - November 27, 2025):

Table 5: Sample Merged Pull Requests (LangChain)

PR #	Title	Date Relative	Date Absolute
#25843	anthropic[patch]: Release 0.41.1	2 days ago	2025-11-25
#25841	docs: Update Anthropic tools example	2 days ago	2025-11-25
#25839	core[patch]: Fix streaming in RunnableBinding	3 days ago	2025-11-24
#25832	community[patch]: Add timeout to Redis	4 days ago	2025-11-23
#25829	docs: Add citation format to papers	5 days ago	2025-11-22

Table 6: Sample Bug Issues (LangChain)

Issue #	Title	Date Relative	Date Absolute
#4521	Bug: Streaming fails with Azure OpenAI	1 day ago	2025-11-26
#4518	Error when using callback with agents	3 days ago	2025-11-24
#4510	Fix: Memory leak in conversation buffer	5 days ago	2025-11-22
#4502	Regression: Tool calling broken in 0.41.0	1 week ago	2025-11-20

D.2 Intel XPU Backend Data

Sample entries from Intel XPU Backend analysis dataset (November 4 - December 2, 2025):

Table 7: Sample Merged Pull Requests (Intel XPU Backend)

PR #	Title	Date Relative	Date Absolute
#2104	Support for Triton 3.2.0	1 day ago	2025-12-01
#2101	Fix GEMM kernel for FP16	2 days ago	2025-11-30
#2098	Add support for grouped convolutions	4 days ago	2025-11-28
#2095	Optimize reduction kernels	6 days ago	2025-11-26
#2092	Fix memory allocation in compiler	1 week ago	2025-11-25

E OpenTelemetry Configuration Files

E.1 Docker Compose Configuration

The Phase 2 OpenTelemetry approach used the following Docker Compose setup:

Listing 11: docker-compose.yml for OTel Stack

```
version: '3.8'

services:
  otel-collector:
    image: otel/opentelemetry-collector-contrib:latest
    container_name: otel-collector
    command: ["--config=/etc/otel-collector-config.yaml"]
    volumes:
      - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml
    ports:
      - "4317:4317"   # OTLP gRPC receiver
```

```
      - "4318:4318"    # OTLP HTTP receiver
      - "8888:8888"    # Prometheus metrics
    networks:
      - otel-network
    depends_on:
      - openobserve

  openobserve:
    image: public.ecr.aws/zinclabs/openobserve:latest
    container_name: openobserve
    ports:
      - "5080:5080"
    environment:
      - ZO_ROOT_USER_EMAIL=admin@example.com
      - ZO_ROOT_USER_PASSWORD=admin123
      - ZO_DATA_DIR=/data
    volumes:
      - openobserve-data:/data
    networks:
      - otel-network

networks:
  otel-network:
    driver: bridge

volumes:
  openobserve-data:
```

E.2 OpenTelemetry Collector Configuration

Listing 12: otel-collector-config.yaml

```yaml
receivers:
  github:
    github_org: langchain-ai
    search_query: "org:langchain-ai is:pr"
    scrapers:
      github:
        collection_interval: 60s
    auth:
      authenticator: bearertokenauth

extensions:
  bearertokenauth:
    token: "${GITHUB_TOKEN}"

processors:
  batch:
    timeout: 10s
    send_batch_size: 100

exporters:
  otlphttp:
    endpoint: "http://openobserve:5080/api/default/"
    headers:
      Authorization: "Basic ${OPENOBSERVE_CREDENTIALS}"
      stream-name: "github-metrics"

```

```
service:
  extensions: [bearertokenauth]
  pipelines:
    metrics:
      receivers: [github]
      processors: [batch]
      exporters: [otlphttp]
```

E.3 Performance Classification Logic

Listing 13: DORA Classification

```python
def classify_deployment_frequency(deployments_per_day):
    """Elite: >1/day, High: weekly, Medium: monthly, Low: <monthly
       """
    if deployments_per_day >= 1:
        return 'Elite'
    elif deployments_per_day >= 1/7:
        return 'High'
    elif deployments_per_day >= 1/30:
        return 'Medium'
    else:
        return 'Low'

def classify_lead_time(median_days):
    """Elite: <1 day, High: 1-7 days, Medium: 1-4 weeks, Low: >1
       month"""
    if median_days < 1:
        return 'Elite'
```

```
    elif median_days <= 7:
        return 'High'
    elif median_days <= 28:
        return 'Medium'
    else:
        return 'Low'

def classify_change_failure_rate(percentage):
    """Elite: 0-15%, High: 16-30%, Medium: 31-45%, Low: >45%"""
    if percentage <= 15:
        return 'Elite'
    elif percentage <= 30:
        return 'High'
    elif percentage <= 45:
        return 'Medium'
    else:
        return 'Low'

def classify_time_to_restore(median_hours):
    """Elite: <1 hour, High: <1 day, Medium: 1-7 days, Low: >1 week
       """
    if median_hours < 1:
        return 'Elite'
    elif median_hours < 24:
        return 'High'
    elif median_hours < 168:  # 7 days
        return 'Medium'
    else:
        return 'Low'
```